Supplementary Materials:
LLM-Assisted Extraction of QSP Calibration Targets

Joel Eliason

Contents

1 S1: Complete SubmodelTarget Example

This section presents a complete calibration target for activated pancreatic stellate cell (aPSC) proliferation, demonstrating all schema components.

1.1 Overview

This target constrains the proliferation rate parameter `k_apsc_prolif` using data from Schneider et al. (2001), who measured PSC proliferation in response to PDGF stimulation via BrdU incorporation assay.

1.2 Complete YAML Specification

```
# Schema: SubmodelTarget
# Target: Activated PSC proliferation rate

target_id: psc_proliferation_PDAC_deriv001

# ============================================================================
# INPUTS: Extracted values with full provenance
# ============================================================================
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                    0.39-fold of control)."

  - name: pdgf_fold_sd
    value: 0.49
    units: dimensionless
    input_type: direct_measurement
    role: auxiliary
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                    0.39-fold of control)."

# ============================================================================
# CALIBRATION: Everything needed for inference
# ============================================================================
calibration:
  parameters:
    - name: k_apsc_prolif
```

```
    units: 1/day
    prior:
      distribution: lognormal
      mu: 0.0      # log(1.0) - expect ~1/day for 4x fold-change in 1 day
      sigma: 1.0
      rationale: |
        Wide log-normal prior centered at 1/day. A 4-fold increase in
        1 day implies k = ln(4) = 1.4/day, so prior median of 1/day
        is reasonable.

state_variables:
  - name: aPSC_rel
    units: dimensionless
    initial_condition:
      value: 1.0
      rationale: |
        PSC proliferation data are reported as fold of control. We
        normalize the initial aPSC population to 1.0 (100% control)
        to match this convention.

model:
  type: exponential_growth
  rate_constant: k_apsc_prolif
  data_rationale: |
    BrdU incorporation assay measures proliferation as fold-change
    over ~24 hours under constant PDGF stimulus. With a single high
    PDGF dose and short duration, exponential growth (dN/dt = k*N)
    fits the data structure.
  submodel_rationale: |
    The full QSPIO_PDAC model has aPSC proliferation via:
    k_apsc_prolif * aPSC * (PDGF / (PDGF_50_prolif + PDGF)).
    At 50 ng/mL PDGF >> PDGF_50_prolif, the saturation term ~ 1,
    reducing to exponential growth with rate k_apsc_prolif.

independent_variable:
  name: time
  units: day
  span: [0.0, 1.0]
  rationale: |
    Proliferation assay duration assumed to be ~24 hours based on
    typical cytokine incubation protocols for rat PSCs.

measurements:
  - name: proliferation_fold_change
    observable:
      type: identity
      state_variables: [aPSC_rel]
      rationale: |
```

```
      The state variable aPSC_rel is the fold-change in aPSC number
      relative to baseline (control = 1.0), which directly matches
      the reported proliferation fold-change.
  units: dimensionless
  uses_inputs:
    - pdgf_fold_mean
    - pdgf_fold_sd
  evaluation_points: [1.0]
  sample_size: 3
  sample_size_rationale: |
    Sample size not explicitly reported; assumed n=3 independent
    experiments, typical for in vitro PSC BrdU proliferation assays.
  distribution_code: |
    def derive_distribution(inputs, ureg):
        import numpy as np

        rng = np.random.default_rng(42)

        mean = inputs['pdgf_fold_mean'].magnitude
        sd = inputs['pdgf_fold_sd'].magnitude
        n = 10000

        # Use lognormal for positive fold-changes
        mu_log = np.log(mean**2 / np.sqrt(mean**2 + sd**2))
        sigma_log = np.sqrt(np.log(1 + sd**2 / mean**2))

        samples = rng.lognormal(mu_log, sigma_log, n)

        median_val = np.median(samples)
        ci_lower = np.percentile(samples, 2.5)
        ci_upper = np.percentile(samples, 97.5)

        return {
            'median': [median_val],
            'ci95': [[ci_lower, ci_upper]],
            'units': 'dimensionless'
        }
  likelihood:
    distribution: lognormal
    rationale: |
      Fold-changes are positive and multiplicative, making
      lognormal appropriate.

identifiability_notes: |
  With a single timepoint (~24h) and normalized initial condition
  (aPSC_rel(0)=1), only k_apsc_prolif is estimated. If substantial
  PSC death occurred over the assay interval, the inferred rate
  would represent net growth rather than pure proliferation.
```

```yaml
# ============================================================================
# EXPERIMENTAL CONTEXT
# ============================================================================
experimental_context:
  species: rat
  system: in_vitro_primary_cells
  indication: PDAC
  cell_types:
    - name: pancreatic stellate cell
      phenotype: activated (alpha-SMA positive)
      isolation_method: Isolated from normal Wistar rat pancreas,
                        activated by culture on plastic
  culture_conditions:
    culture_type: 2d_monolayer

# ============================================================================
# STUDY INTERPRETATION
# ============================================================================
study_interpretation: |
  Schneider et al. (2001) measured proliferation of cultured rat
  pancreatic stellate cells (PSCs) in response to growth factors using
  BrdU incorporation. Addition of 50 ng/mL PDGF increased PSC
  proliferation to 4.37 +/- 0.49-fold of control, indicating that PDGF
  acts as a strong mitogen for activated PSCs. We interpret the
  50 ng/mL PDGF condition as approximating a near-maximal PDGF stimulus
  and use the reported fold-change over ~24 hours as a quantitative
  constraint on the maximal PDGF-driven proliferation rate parameter
  k_apsc_prolif.

key_assumptions:
  - |
    50 ng/mL PDGF corresponds to a near-saturating concentration for
    PSC proliferation, so PDGF/(PDGF_50_prolif + PDGF) ~ 1.
  - |
    The proliferation assay duration is approximately 24 hours,
    consistent with typical cytokine incubation protocols.
  - |
    BrdU incorporation fold-change is proportional to fold-change in
    PSC cell number (negligible death over 24 hours).
  - |
    Rat activated PSC proliferation kinetics in vitro provide an upper
    bound for human tumor aPSC proliferation in vivo.

key_study_limitations:
  - |
    Proliferation was measured via BrdU incorporation rather than
    direct cell counts.
```

```
  - |
    Single PDGF concentration and single assay interval constrain
    only the maximal rate.
  - |
    Data from rat PSCs in vitro, not human PDAC-associated stellate cells.
  - |
    Background serum proliferative signals not explicitly modeled.

# ============================================================================
# DATA SOURCES
# ============================================================================
primary_data_source:
  doi: "10.1152/ajpcell.2001.281.2.C532"
  title: "Identification of mediators stimulating proliferation and
          matrix synthesis of rat pancreatic stellate cells"
  authors: [Schneider, Schmid, Liptay, Schmid, Pohl]
  year: 2001
  source_tag: Schneider2001_PSC_PDGF

secondary_data_sources:
  - doi: "10.1136/gut.44.4.534"
    title: "Pancreatic stellate cells are activated by proinflammatory
            cytokines: implications for pancreatic fibrogenesis"
    authors: [Apte, Haber, Darby, Rodgers, McCaughan, Korsten, Pirola, Wilson]
    year: 1999
    source_tag: Apte1999_PSC_Cytokines
    contribution: |
      Provides context on PSC activation and cytokine response protocols.
```

1.3 Schema Structure Summary

The SubmodelTarget schema has the following top-level structure:

- **target_id**: Unique identifier for the calibration target
- **inputs**: List of values extracted from literature with provenance
- **calibration**: Everything needed for inference code generation
  - `parameters`: Parameters to estimate with priors
  - `state_variables`: ODE state variables with initial conditions
  - `model`: Model type specification (built-in or custom)
  - `independent_variable`: Time or other independent variable
  - `measurements`: Observables with likelihoods
  - `identifiability_notes`: Discussion of parameter identifiability
- **experimental_context**: Species, system, cell types, culture conditions
- **study_interpretation**: Scientific interpretation of the data

- **key_assumptions**: Assumptions made in using this data
- **key_study_limitations**: Known limitations of the source study
- **primary_data_source**: Primary literature reference with DOI
- **secondary_data_sources**: Additional references

2 S2: Supported Model Types

The SubmodelTarget schema supports 15 built-in forward model types, organized into four categories.

Table 1: Built-in forward model types organized by category.

Category	Model Type	Formulation	Key Fields
ODE templates	`exponential_growth`	$dy/dt = k \cdot y$	`rate_constant`
	`first_order_decay`	$dy/dt = -k \cdot y$	`rate_constant`
	`logistic`	$dy/dt = k \cdot y(1 - y/K)$	`rate_constant`, `carrying_cap`
	`saturation`	$dy/dt = k(1 - y)$	`rate_constant`
	`two_state`	$dA/dt = -kA$, $dB/dt = kA$	`forward_rate`
	`michaelis_menten`	$dy/dt = -V_{\max} y/(K_m + y)$	`vmax`, `km`
Structured steady-state	`steady_state_density`	rate = density × loss	`target_rate`, `observed_densi`
	`steady_state_fraction`	rate = $f \times \rho_{\text{parent}} \times$ loss	`target_rate`, `observed_fract`
	`steady_state_concentration`	sec = $C \times k_{\text{cl}} \times V$	`target_rate`, `observed_conce`
	`steady_state_ratio`	$r = k_1/k_2$	`target_rate`, `observed_ratio`
	`steady_state_proliferation`	$f = k_p/(k_p + k_d)$	`target_rate`, `observed_fract`
Accumulation	`batch_accumulation`	$C = (\text{sec} \times n \times t)/V$	`target_rate`, `cell_count`, `inc`
Generic/ fallback	`algebraic`	(closed-form formula)	`code`, `code_julia`
	`direct_fit`	(curve fitting)	`curve`
	`custom`	(user-provided ODE)	`code`, `code_julia`

Each structured steady-state and accumulation model declares its fields as typed roles: a field can reference a calibration parameter (to be estimated), an extracted input (from the inputs layer), a curated reference value from a shared database, or a numeric literal. This role-based design allows the same template to serve different estimation scenarios depending on which field is treated as the unknown.

3 S3: Validation Checks

The validation framework includes ten automated checks implemented as Pydantic `@model_validator` methods. Each validator runs after field parsing and raises a `ValidationError` with detailed messages when constraints are violated.

3.1 Summary of Validators

1. **Input reference validation**: All `uses_inputs` references match defined input names

2. **Source reference validation**: All `source_ref` values match defined source tags
3. **Parameter reference validation**: Model parameter roles reference defined parameters
4. **State variable reference validation**: Observable `state_variables` match defined state variables
5. **DOI resolution**: All DOIs resolve via CrossRef API
6. **Value-in-snippet validation**: Extracted values appear verbatim in quoted snippets
7. **Unit validation**: All unit strings parse with Pint
8. **Code syntax validation**: Custom code passes Python AST parsing
9. **Span ordering validation**: Time spans have start $<$ end
10. **ODE model requirements**: ODE models have required state variables and spans

3.2 DOI Resolution Validator

The DOI validator queries the CrossRef API to verify that every cited DOI resolves to a valid publication. This catches fabricated citations before they enter the calibration pipeline. The validator also performs fuzzy matching on title and year to detect metadata inconsistencies.

```
def resolve_doi(doi: str) -> Optional[dict]:
    """Resolve DOI and get metadata from CrossRef."""
    doi_clean = doi.replace("https://doi.org/", "").strip()

    url = f"https://doi.org/{doi_clean}"
    headers = {"Accept": "application/vnd.citationstyles.csl+json"}
    response = requests.get(url, headers=headers, timeout=10)

    if response.status_code != 200:
        return None  # DOI does not resolve

    metadata = response.json()
    return {
        "title": metadata.get("title", ""),
        "first_author": metadata.get("author", [{}])[0].get("family", ""),
        "year": metadata.get("issued", {}).get("date-parts", [[]])[0][0]
    }

@model_validator(mode="after")
def validate_doi_resolution_and_metadata(self) -> "SubmodelTarget":
    """Validate that DOIs resolve and metadata matches."""
    errors = []

    metadata = resolve_doi(self.primary_data_source.doi)
    if metadata is None:
        errors.append(f"DOI '{self.primary_data_source.doi}' failed to resolve")
```

```
    else:
        # Check title match (fuzzy)
        if not fuzzy_match(self.primary_data_source.title, metadata["title"]):
            errors.append(f"Title mismatch with CrossRef")
        # Check year match
        if self.primary_data_source.year != metadata["year"]:
            errors.append(f"Year mismatch: {self.primary_data_source.year} vs {metadata['year']

    if errors:
        raise ValueError("DOI validation errors:\n  - " + "\n  - ".join(errors))
    return self
```

Example failure: If an LLM fabricates a citation with DOI `10.1234/fake.2023.12345`, the CrossRef query returns a 404 error and validation fails with:

```
ValidationError: DOI '10.1234/fake.2023.12345' failed to resolve.
Verify at https://doi.org/10.1234/fake.2023.12345
```

3.3 Value-in-Snippet Validator

The value-in-snippet validator checks that each extracted numeric value appears in its associated source text. This catches hallucinations where an LLM extracts a plausible but incorrect value. The validator handles multiple numeric formats including scientific notation, percentages, and Unicode superscripts.

```
def check_value_in_text(text: str, value: float) -> bool:
    """Check if numeric value appears in text."""
    text_norm = text.lower().replace(",", "")
    patterns = []

    # Direct value and common formats
    patterns.append(str(value))
    patterns.append(f"{value:.1f}")
    patterns.append(f"{value:.2f}")

    # Scientific notation (e.g., "1.5e-3", "1.5E-03")
    if abs(value) < 0.01 or abs(value) > 1000:
        patterns.append(f"{value:e}")
        patterns.append(f"{value:.2e}")

    # Percentage format (if value is between 0 and 1)
    if 0 < value < 1:
        pct = value * 100
        patterns.extend([f"{pct}%", f"{pct:.0f}%", f"{pct:.1f}%"])

    return any(p.lower() in text_norm for p in patterns)

@model_validator(mode="after")
def validate_input_values_in_snippets(self) -> "SubmodelTarget":
```

```
    """Validate that extracted values appear in their value_snippet."""
    errors = []
    for inp in self.inputs:
        if not inp.value_snippet:
            continue
        # Skip types that may not have explicit values in text
        if inp.input_type in (InputType.INFERRED_ESTIMATE, InputType.ASSUMED_VALUE):
            continue

        if not check_value_in_text(inp.value_snippet, inp.value):
            errors.append(
                f"Input '{inp.name}': value {inp.value} not found in snippet "
                f"'{inp.value_snippet[:60]}...'"
            )

    if errors:
        raise ValueError("Value-snippet mismatches (possible hallucination):\n  - "
                         + "\n  - ".join(errors))
    return self
```

Example failure: If an input claims `value: 4.37` but the snippet reads "Cell proliferation was 3.21-fold of control", validation fails with:

```
ValidationError: Value-snippet mismatches (possible hallucination):
  - Input 'pdgf_fold_mean': value 4.37 not found in snippet
    'Cell proliferation was 3.21-fold of control...'
```

This validator is particularly effective because LLMs that hallucinate values rarely fabricate consistent snippets—the mismatch provides a reliable detection signal.

3.4 Unit Validation

The unit validator parses all unit strings using Pint, a Python library for physical quantities. Malformed or unrecognized units trigger validation failure.

```
@model_validator(mode="after")
def validate_units_are_valid_pint(self) -> "SubmodelTarget":
    """Validate that all unit strings are valid Pint units."""
    from pint import UnitRegistry
    ureg = UnitRegistry()
    errors = []

    def check_unit(unit_str: str, location: str):
        try:
            ureg(unit_str)
        except Exception as e:
            errors.append(f"{location}: '{unit_str}' is not valid ({e})")

    for inp in self.inputs:
```

```
        check_unit(inp.units, f"Input '{inp.name}'")
    for param in self.calibration.parameters:
        check_unit(param.units, f"Parameter '{param.name}'")

    if errors:
        raise ValueError("Invalid Pint units:\n  - " + "\n  - ".join(errors))
    return self
```

Example failure: If a parameter specifies `units: cells/mL/day` (invalid compound unit), validation fails with:

```
ValidationError: Invalid Pint units:
  - Parameter 'k_recruit': 'cells/mL/day' is not valid (...)
```

3.5 Reference Consistency Validators

Four validators check that all cross-references within the schema are consistent:

- **Input references**: Every `uses_inputs` entry and `initial_condition.input_ref` must match a defined input name.
- **Source references**: Every `source_ref` must match `primary_data_source.source_tag` or a `secondary_data_sources[].source_tag`.
- **Parameter references**: When a model field like `rate_constant` contains a string (not an `InputRef`), it must match a parameter name in `calibration.parameters`.
- **State variable references**: Every state variable referenced in `observable.state_variables` must exist in `calibration.state_variables`.

These validators catch internal inconsistencies where an LLM generates references to entities that don't exist elsewhere in the document.

4 S4: Generated Julia Code Structure

The translator generates Julia scripts with the following structure:

```
using DifferentialEquations
using Turing
using Distributions

# Observed data (from inputs layer)
const obs_proliferation_median = 4.37
const obs_proliferation_sigma = 0.25  # derived from CI95

# ODE function
function ode_proliferation!(du, u, p, t)
    k = p[1]
    du[1] = k * u[1]  # exponential growth
end
```

```
# Simulate function
function simulate_proliferation(k; t_span=(0.0, 1.0), u0=[1.0])
    prob = ODEProblem(ode_proliferation!, u0, t_span, [k])
    sol = solve(prob, Tsit5())
    return sol[end][1]  # final value
end

# Turing model
@model function proliferation_model()
    # Prior
    k_apsc_prolif ~ LogNormal(0.0, 1.0)

    # Simulate
    pred = simulate_proliferation(k_apsc_prolif)

    # Likelihood
    obs_proliferation_median ~ LogNormal(log(pred), obs_proliferation_sigma)
end

# Run inference
model = proliferation_model()
chain = sample(model, NUTS(), MCMCThreads(), 1000, 4)
```

For joint inference over multiple targets, the translator identifies shared parameters by name and generates a single model with one prior per unique parameter.

5 S5: Pydantic Model Definitions

This section provides the key Pydantic model definitions that implement both calibration target schemas. The full implementations are available in the repository at `src/qsp_llm_workflows/core/calibration/`.

5.1 Top-Level Model

The `SubmodelTarget` class is the entry point for validation. When a YAML file is loaded, Pydantic recursively validates all nested models.

```
class SubmodelTarget(BaseModel):
    """
    Submodel-based calibration target for QSP parameter inference.

    Separates:
    - ‘inputs‘: What was extracted from papers (with full provenance)
    - ‘calibration‘: How to use those inputs for inference
    """
    target_id: str = Field(description="Unique identifier for this target")

    # Extracted data with provenance
```

```
    inputs: List[Input] = Field(
        description="Values extracted from literature with full provenance"
    )

    # Calibration specification
    calibration: Calibration = Field(
        description="Everything needed for inference code generation"
    )

    # Experimental context
    experimental_context: ExperimentalContext = Field(
        description="Experimental context for the target"
    )

    # Study interpretation
    study_interpretation: str = Field(
        description="Scientific interpretation of how the study data informs the model"
    )
    key_assumptions: List[str] = Field(
        description="Key assumptions made in using this data"
    )
    key_study_limitations: Optional[List[str]] = Field(
        default=None, description="Known limitations of the study"
    )

    # Data sources
    primary_data_source: PrimaryDataSource = Field(
        description="Primary literature source"
    )
    secondary_data_sources: Optional[List[SecondaryDataSource]] = Field(
        default=None, description="Additional literature sources"
    )

    # Validators (10 total) defined via Pydantic decorators
    # See Section S3 for implementation details
```

5.2 Input Model

Each input captures a single value extracted from literature with full provenance:

```
class InputType(str, Enum):
    """Type of input extracted from literature."""
    DIRECT_MEASUREMENT = "direct_measurement"   # Value reported directly
    PROXY_MEASUREMENT = "proxy_measurement"     # Requires conversion
    EXPERIMENTAL_CONDITION = "experimental_condition"  # Protocol choice
    INFERRED_ESTIMATE = "inferred_estimate"     # Interpreted from qualitative text
    ASSUMED_VALUE = "assumed_value"             # Domain knowledge, not in source
```

```
class InputRole(str, Enum):
    """Role of input in calibration."""
    INITIAL_CONDITION = "initial_condition"  # IC for ODE integration
    TARGET = "target"                        # Calibration target (likelihood term)
    FIXED_PARAMETER = "fixed_parameter"      # Fixed value in model
    AUXILIARY = "auxiliary"                  # Supporting data

class Input(BaseModel):
    """A value extracted from literature with full provenance."""
    name: str = Field(description="Unique identifier for this input")
    value: float = Field(description="Extracted numeric value")
    units: str = Field(description="Units of the value")
    uncertainty: Optional[Uncertainty] = Field(default=None)
    n: Optional[int] = Field(default=None, description="Sample size")
    input_type: InputType
    role: Optional[InputRole] = Field(default=None)
    source_ref: str = Field(description="Reference to source_tag in data sources")
    source_location: str = Field(description="Location within the source")
    value_snippet: Optional[str] = Field(
        default=None,
        description="Exact text from paper containing the value (for validation)"
    )
```

5.3 Model Type Discriminated Union

The schema uses Pydantic's discriminated union pattern to support multiple model types, each with type-specific required fields:

```
class BaseModelSpec(BaseModel):
    """Base class for all model specifications."""
    data_rationale: str = Field(
        description="Why this model type fits the experimental data"
    )
    submodel_rationale: str = Field(
        description="Why this is a valid submodel of the full QSP model"
    )

class ExponentialGrowthModel(BaseModelSpec):
    """Exponential growth: dy/dt = k * y"""
    type: Literal["exponential_growth"] = "exponential_growth"
    rate_constant: ParameterRole = Field(
        description="Rate constant parameter name or input_ref"
    )

class TwoStateModel(BaseModelSpec):
    """Two-state transition: A -> B with first-order kinetics."""
    type: Literal["two_state"] = "two_state"
    forward_rate: ParameterRole = Field(
```

```
        description="Forward transition rate constant"
    )

class CustomModel(BaseModelSpec):
    """Custom ODE with user-provided code"""
    type: Literal["custom"] = "custom"
    code: str = Field(description="Python ODE function")
    code_julia: str = Field(description="Julia ODE function for inference")

# Discriminated union selects model class based on 'type' field
Model = Annotated[
    Union[
        FirstOrderDecayModel,
        ExponentialGrowthModel,
        LogisticModel,
        MichaelisMentenModel,
        TwoStateModel,
        SaturationModel,
        DirectConversionModel,
        DirectFitModel,
        CustomModel,
    ],
    Field(discriminator="type"),
]
```

The `discriminator="type"` annotation tells Pydantic to inspect the `type` field in the YAML to determine which model class to instantiate. This ensures that an `exponential_growth` model is validated against `ExponentialGrowthModel` (which requires `rate_constant`) while a `two_state` model is validated against `TwoStateModel` (which requires `forward_rate`).

5.4 Prior Specification

Priors are specified with distribution type and parameters:

```
class PriorDistribution(str, Enum):
    """Supported prior distribution types."""
    LOGNORMAL = "lognormal"    # For positive parameters (rates, densities)
    NORMAL = "normal"          # For unconstrained parameters
    UNIFORM = "uniform"        # For bounded parameters
    HALF_NORMAL = "half_normal"  # For positive parameters with mode at 0

class Prior(BaseModel):
    """Prior distribution specification for Bayesian inference."""
    distribution: PriorDistribution
    mu: Optional[float] = Field(default=None, description="Location parameter")
    sigma: Optional[float] = Field(default=None, description="Scale parameter")
    lower: Optional[float] = Field(default=None, description="Lower bound")
    upper: Optional[float] = Field(default=None, description="Upper bound")
```

```
    rationale: Optional[str] = Field(default=None)

    @model_validator(mode="after")
    def validate_prior_params(self) -> "Prior":
        """Validate that required parameters are provided for each distribution."""
        if self.distribution == PriorDistribution.LOGNORMAL:
            if self.mu is None or self.sigma is None:
                raise ValueError("lognormal prior requires mu and sigma")
        elif self.distribution == PriorDistribution.UNIFORM:
            if self.lower is None or self.upper is None:
                raise ValueError("uniform prior requires lower and upper")
            if self.lower >= self.upper:
                raise ValueError("uniform lower must be < upper")
        return self
```

5.5 Loading and Validating YAML Files

To load and validate a calibration target:

```
import yaml
from qsp_llm_workflows.core.calibration import SubmodelTarget

# Load YAML file
with open("psc_proliferation_PDAC_deriv001.yaml") as f:
    data = yaml.safe_load(f)

# Parse and validate (raises ValidationError on failure)
target = SubmodelTarget(**data)

# Access validated fields
print(f"Target: {target.target_id}")
print(f"Parameters: {[p.name for p in target.calibration.parameters]}")
print(f"Model type: {target.calibration.model.type}")
```

If validation fails, Pydantic raises a `ValidationError` with detailed error messages:

```
pydantic_core._pydantic_core.ValidationError: 2 validation errors for SubmodelTarget
calibration.measurements.0.uses_inputs.0
  Input 'typo_input_name' not found in inputs [...]
primary_data_source.doi
  DOI '10.1234/fake' failed to resolve [...]
```

5.6 CalibrationTarget Model

The `CalibrationTarget` schema represents clinical and in vivo endpoints for full-model calibration. Where SubmodelTarget isolates a single mechanism, CalibrationTarget operates on the full model state.

```
class CalibrationTarget(BaseModel):
```

```
    """
    Calibration target for full-model Bayesian inference.

    Represents a clinical or in vivo measurement that constrains
    model behavior at the system level.
    """
    # Observable: how to compute the measurement from model species
    observable: Observable = Field(
        description="Maps full model species to measured quantity"
    )

    # Empirical data: literature-derived distribution
    empirical_data: CalibrationTargetEstimates = Field(
        description="Observed values derived from literature via Monte Carlo"
    )

    # Clinical/experimental context
    experimental_context: ExperimentalContext = Field(
        description="Species, system, indication, treatment, staging"
    )
    scenario: Optional[Scenario] = Field(
        default=None,
        description="Treatment scenario for intervention studies"
    )

    # Interpretation and provenance (shared with SubmodelTarget)
    study_interpretation: str
    key_assumptions: List[str]
    key_study_limitations: Optional[List[str]] = None
    primary_data_source: PrimaryDataSource
    secondary_data_sources: Optional[List[SecondaryDataSource]] = None
    source_relevance: Optional[SourceRelevanceAssessment] = None

    # 30+ validators (see Section S3)
```

5.7 Observable

The `Observable` specifies how to compute the experimental measurement from full model species. Unlike SubmodelTarget's forward model (which defines isolated dynamics), the observable operates on the solved full-model state.

```
class Observable(BaseModel):
    """
    Full-model observable for CalibrationTarget.

    The code field contains a Python function that computes the
    measured quantity from model species at each time point.
    """
```

```
    code: str = Field(
        description="Python: compute_observable(time, species_dict, "
                    "constants, ureg) -> Quantity array"
    )
    units: str = Field(description="Units of the observable output")
    species: List[str] = Field(
        description="Model species accessed (e.g., ['V_T.CD8', 'V_T.C1'])"
    )
    constants: Optional[List[ObservableConstant]] = Field(
        default=None,
        description="Named constants with provenance"
    )
    inputs: Optional[List[SubmodelInput]] = Field(
        default=None,
        description="Literature values used in the observable"
    )
    support: SupportType = Field(
        description="Mathematical domain: positive, non_negative, "
                    "unit_interval, real"
    )
    mapping_rationale: str = Field(
        description="How the literature measurement maps to model species"
    )
    # Denominator audit fields for density/fraction observables
    experimental_denominator: Optional[str] = None
    model_denominator_species: Optional[List[str]] = None
```

Each `ObservableConstant` requires a `biological_basis` explanation and must trace to either the curated reference database or a specific literature source.

5.8 CalibrationTargetEstimates

The empirical data block derives summary statistics from literature inputs via Monte Carlo simulation.

```
class CalibrationTargetEstimates(BaseModel):
    """
    Empirical data block: literature values -> distribution for likelihood.
    """
    median: List[float] = Field(
        description="Point estimates (length 1 for scalar, "
                    "or matching index_values for vector)"
    )
    ci95: List[List[float]] = Field(
        description="95% credible intervals [[lo, hi], ...]"
    )
    units: str
    sample_size: Union[int, List[int]]
```

```
    sample_size_rationale: str

    # For vector-valued data (time courses, dose responses)
    index_values: Optional[List[float]] = None
    index_unit: Optional[str] = None
    index_type: Optional[IndexType] = None

    # Inputs and assumptions
    inputs: List[EstimateInput] = Field(
        description="Literature values with provenance"
    )
    assumptions: Optional[List[ModelingAssumption]] = None

    # Monte Carlo code
    distribution_code: str = Field(
        description="Python: derive_distribution(inputs, ureg) -> "
                    "{'median': [...], 'ci95': [...], 'units': str}"
    )
```

5.9 Scenario and Intervention

For studies involving treatment, the `Scenario` captures the clinical context.

```
class Intervention(BaseModel):
    """A single treatment intervention."""
    agent: str = Field(description="Drug or treatment name")
    dose: Optional[str] = None
    route: Optional[str] = None
    schedule: Optional[str] = None

class Scenario(BaseModel):
    """Treatment scenario for intervention studies."""
    interventions: List[Intervention]
    treatment_line: Optional[str] = None
    prior_treatments: Optional[List[str]] = None
    description: str = Field(
        description="Narrative description of the clinical scenario"
    )
```

6 S6: SubmodelTarget Schema Details

This section provides detailed field descriptions and representative YAML examples for each layer of the SubmodelTarget schema. These examples are referenced from the Methods section; the complete schema is defined in Section **??**.

6.1 S6.1: Input Fields

Each input captures a single value extracted from literature with full provenance. The key fields are:

- `name`: Unique identifier referenced elsewhere in the schema
- `value` and `units`: The extracted numeric value with units
- `uncertainty`: Standard deviation or 95% confidence interval, when reported
- `n`: Sample size, for propagating uncertainty through the likelihood
- `input_type`: Classification as `direct_measurement`, `proxy_measurement`, `inferred_estimate`, or `assumed_value`
- `role`: How the input is used: `target` (likelihood term), `auxiliary` (supporting), `initial_condition`, or `fixed_parameter`
- `source_ref`: Reference to a literature source with DOI
- `value_snippet`: Quoted text from the source containing the value

Code block **??** shows a representative input.

6.2 S6.2: Parameter and Prior Specification

Each parameter includes a prior distribution with documented rationale. The schema supports four distribution families: LogNormal (positive parameters with multiplicative uncertainty), Normal (unconstrained), Uniform (bounded with no preference), and HalfNormal (positive with mode at zero).

Code block **??** shows a parameter specification.

6.3 S6.3: Forward Model Specification

The forward model captures the dynamics relevant to the experiment. For time-course data, this is typically a simplified ODE; for steady-state or derived measurements, an algebraic model may suffice.

Code block **??** shows a forward model using the exponential growth template.

6.4 S6.4: Error Model Specification

The error model specifies the statistical relationship between predictions and observed data. The `observation_code` derives the observed value and uncertainty from raw inputs. The `observable` defines the transformation from state variables to predicted measurement. The `likelihood` specifies the distribution family.

Code block **??** shows a complete error model entry.

7 S7: Complete CalibrationTarget Example

This section presents a complete CalibrationTarget for baseline CD8+ T cell density in PDAC, demonstrating how full-model observables, reference database constants, and Monte Carlo distribution derivation work together.

```yaml
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    uncertainty:
      sd: 0.49
    n: 3
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49-
                    and 2.96 +/- 0.39-fold of control)."
```

Figure 1: An input capturing a proliferation measurement with provenance.

```yaml
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      # log(1.0)
        sigma: 1.0
        rationale: |
          Wide prior centered at 1/day. A 4-fold increase
          in 1 day implies k ~ ln(4) ~ 1.4/day.
```

Figure 2: A parameter with lognormal prior and documented rationale.

```
calibration:
  forward_model:
    type: exponential_growth
    rate_constant: k_apsc_prolif
    independent_variable:
      name: time
      units: day
      span: [0.0, 1.0]
    state_variables:
      - name: aPSC_rel
        units: dimensionless
        initial_condition:
          value: 1.0
          rationale: Normalized to control baseline.
    data_rationale: |
      BrdU assay measures proliferation over ~24h with
      constant PDGF stimulus. Exponential growth fits.
    submodel_rationale: |
      At saturating PDGF, the full model reduces to
      dN/dt = k_apsc_prolif * N.
```

Figure 3: Forward model specification with state variable and time span.

```
  error_model:
    - name: viability_48h
      units: dimensionless
      uses_inputs: [viability_mean, viability_sem]
      evaluation_points: [2.0]  # days
      sample_size: 3
      observable:
        type: identity
        state_variables: [aPSC_norm]
      observation_code: |
        def derive_observation(inputs, sample_size):
            mean = inputs['viability_mean']
            sem = inputs['viability_sem']
            sd = sem * np.sqrt(sample_size)  # SEM to SD
            return {'value': mean, 'sd': sd}
      likelihood:
        distribution: normal
        rationale: |
          Viability near 1 with small SD; normal adequate.
```

Figure 4: Error model specification linking forward model predictions to observed data.

7.1 Overview

This target constrains baseline intratumoral CD8+ T cell density using immunohistochemistry data from Jansen et al. (2021), who measured CD8 density across 599 pancreatic cancers using tissue microarrays. The observable computes CD8+ density from full model species (effector and exhausted CD8+ T cells) and reference database constants (cancer cell geometry). The distribution code combines two patient subgroups via a mixture of lognormals.

7.2 Observable

The observable maps full model species to the experimental measurement:

```
observable:
  code: |
    def compute_observable(time, species_dict, constants, ureg):
        import numpy as np
        cd8_eff = species_dict['V_T.CD8']
        cd8_exh = species_dict['V_T.CD8_exh']
        C1 = species_dict['V_T.C1']
        cross_section = constants['pdac_cancer_cell_cross_section']
        cellularity = constants['pdac_cellularity_fraction']

        total_cd8 = cd8_eff + cd8_exh
        tumor_area = C1 * cross_section / cellularity
        density = total_cd8 / tumor_area
        return density * ureg('cell / millimeter**2')
  units: cell / millimeter**2
  species: [V_T.CD8, V_T.CD8_exh, V_T.C1]
  constants:
    - name: pdac_cancer_cell_cross_section
      value: 0.000227
      units: millimeter**2 / cell
      biological_basis: |
        Mean cross-sectional area of PDAC cancer cells from
        histomorphometric measurements. Derived from mean cell
        diameter of 17 micrometers.
      source_type: reference_db
      reference_db_name: pdac_cancer_cell_cross_section
    - name: pdac_cellularity_fraction
      value: 0.15
      units: dimensionless
      biological_basis: |
        Fraction of tumor volume occupied by cancer cells in PDAC.
        PDAC tumors are characteristically desmoplastic with low
        cellularity (10-20%).
      source_type: reference_db
      reference_db_name: pdac_cellularity_fraction
  support: positive
  mapping_rationale: |
```

```
    IHC measures CD8+ cells per mm^2 of tissue section. The model
    tracks CD8+ T cells as counts in the tumor compartment (V_T).
    To convert model cell counts to density, we divide by tumor
    area estimated from cancer cell count, cross-section, and
    cellularity fraction.
  experimental_denominator: "mm^2 of tumor tissue section area"
  model_denominator_species: [V_T.C1]
```

7.3 Empirical Data

The distribution code combines two patient subgroups (DOG1-negative and DOG1-positive) via mixture-of-lognormals Monte Carlo:

```
empirical_data:
  median: [138.8]
  ci95: [[20.3, 965.2]]
  units: cell / millimeter**2
  sample_size: 444
  sample_size_rationale: |
    Total of 444 patients: 368 DOG1-negative + 76 DOG1-positive.
  inputs:
    - name: mean_cd8_stroma_neg
      value: 227.7
      units: cell / millimeter**2
      source_ref: Jansen2021
      source_location: "Table 4"
      value_snippet: "CD8 ... 227.7 +/- 15.0"
    - name: sem_cd8_stroma_neg
      value: 15.0
      units: cell / millimeter**2
      source_ref: Jansen2021
      source_location: "Table 4"
      value_snippet: "CD8 ... 227.7 +/- 15.0"
    - name: n_stroma_neg
      value: 368
      units: dimensionless
      source_ref: Jansen2021
      source_location: "Table 1"
      value_snippet: "DOG1 negative 368 (61.4%)"
    - name: mean_cd8_stroma_pos
      value: 220.1
      units: cell / millimeter**2
      source_ref: Jansen2021
      source_location: "Table 4"
      value_snippet: "CD8 ... 220.1 +/- 33.0"
    - name: sem_cd8_stroma_pos
      value: 33.0
      units: cell / millimeter**2
```

```
    source_ref: Jansen2021
    source_location: "Table 4"
    value_snippet: "CD8 ... 220.1 +/- 33.0"
  - name: n_stroma_pos
    value: 76
    units: dimensionless
    source_ref: Jansen2021
    source_location: "Table 1"
    value_snippet: "DOG1 positive 76 (12.7%)"
distribution_code: |
  def derive_distribution(inputs, ureg):
      import numpy as np
      rng = np.random.default_rng(42)
      n_mc = 100000

      # Subgroup 1: DOG1-negative
      mean1 = inputs['mean_cd8_stroma_neg'].magnitude
      sem1 = inputs['sem_cd8_stroma_neg'].magnitude
      n1 = int(inputs['n_stroma_neg'].magnitude)
      sd1 = sem1 * np.sqrt(n1)

      # Subgroup 2: DOG1-positive
      mean2 = inputs['mean_cd8_stroma_pos'].magnitude
      sem2 = inputs['sem_cd8_stroma_pos'].magnitude
      n2 = int(inputs['n_stroma_pos'].magnitude)
      sd2 = sem2 * np.sqrt(n2)

      # Convert to lognormal parameters
      def to_lognormal(m, s):
          var = s**2
          mu = np.log(m**2 / np.sqrt(m**2 + var))
          sigma = np.sqrt(np.log(1 + var / m**2))
          return mu, sigma

      mu1, sig1 = to_lognormal(mean1, sd1)
      mu2, sig2 = to_lognormal(mean2, sd2)

      # Mixture sampling
      w1 = n1 / (n1 + n2)
      mask = rng.random(n_mc) < w1
      samples = np.where(
          mask,
          rng.lognormal(mu1, sig1, n_mc),
          rng.lognormal(mu2, sig2, n_mc)
      )

      return {
          'median': [float(np.median(samples))],
```

```
            'ci95': [[float(np.percentile(samples, 2.5)),
                      float(np.percentile(samples, 97.5))]],
            'units': 'cell / millimeter**2'
        }
```

7.4 Source and Context

```
primary_data_source:
  doi: "10.7717/peerj.11397"
  title: ""; expression of immune cell markers in the tumor
          microenvironment and association with survival"
  authors: [Jansen, van Doorn,"; et al"]
  year: 2021
  source_tag: Jansen2021

experimental_context:
  species: human
  system: clinical_resection
  indication: PDAC
  treatment_status: treatment_naive
  stage: resectable

study_interpretation: |
  Jansen et al. measured CD8+ T cell density across 599 pancreatic
  cancers using tissue microarrays and digital image analysis. We
  combine two subgroups (DOG1-positive and DOG1-negative) as a
  mixture of lognormals to capture the full population distribution.

key_assumptions:
  - "CD8 IHC detects both effector and exhausted T cells equally"
  - "Reported +/- values are SEM (confirmed by sqrt(n) test)"
  - "DOG1 expression does not significantly affect CD8 density"

key_study_limitations:
  - "TMA cores (0.6mm) may not represent whole-slide heterogeneity"
  - "Mixed pathologic stages without neoadjuvant therapy reporting"
  - "2D histology section vs. 3D model compartment"
```

8 S8: CalibrationTarget Detailed Metrics

This section provides detailed metrics for the 50 CalibrationTargets applied to the PDAC QSP model.

Table 2: CalibrationTarget observable categories.

Observable Category	Count
Immune/stromal fractions	27
Cell densities	8
Population ratios	8
Clinical endpoints	3
Fold-changes	2
Metabolic/TME markers	2

Table 3: Observable complexity across 50 CalibrationTargets.

Metric	Mean	Min	Max
Model species per observable	7.8	1	19
Named constants per observable	0.7	0	5
Empirical inputs per target	6.0	1	18

8.1 Observable Categories

8.2 Observable Complexity

The most complex observables (19 species) are neoadjuvant immunotherapy targets computing immune subset percentages of all nucleated cells, requiring all modeled immune, stromal, and tumor populations in the denominator. The highest-constant observables (5 constants) require geometric and cellularity reference parameters for converting between model cell counts and histologic measurements. Across all targets, observables reference 25 unique model species spanning immune cells ($CD8^+$ effector and exhausted, Treg, Th, NK), myeloid cells (M1/M2 macrophages, MDSC, cDC1/cDC2), stromal cells (myCAF, iCAF, apCAF, quiescent PSC), tumor cells, ECM, and microenvironment variables (VEGF, lactate, pO2, glucose, HIF).

8.3 Extraction Model Attribution

Table 4: CalibrationTarget extraction model breakdown.

Extraction Model	Count	Percentage
Claude Opus 4	27	54%
GPT-5.1	22	44%
Manually curated	1	2%
Total	**50**	

Of the 49 AI-generated targets, 30 (60%) are tagged as human-verified, indicating that a domain expert reviewed the complete target after initial extraction. 15 targets (30%) include data digitized from figures, where per-patient data points were manually extracted and the CalibrationTarget structure was assembled by the LLM from those data points.

8.4 Primary Data Sources

The 50 targets drew from 21 unique primary sources (by DOI), with publication years spanning 2000–2026. Two sources contributed the majority of targets: a neoadjuvant immunotherapy trial (Li et al., Cancer Cell, 2022) provided data for all 18 GVAX/nivolumab targets, and a large-scale immunohistochemistry study (Liudahl et al., Cancer Discovery, 2021) provided data for 11 baseline targets covering immune cell densities, fractions, and ratios.